



◆ CS EDUCATION ◆



프로세스와 스레드



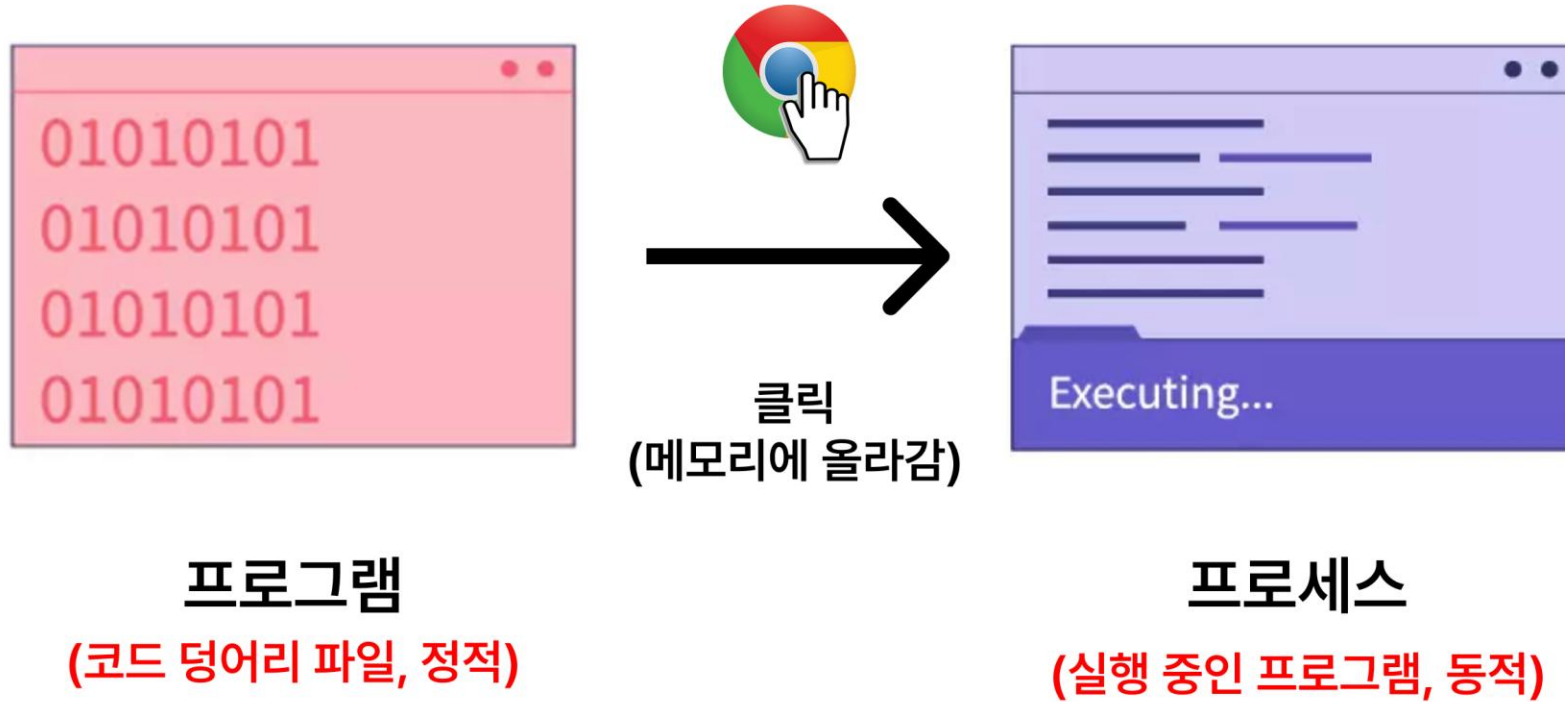
25.11.14
우재원



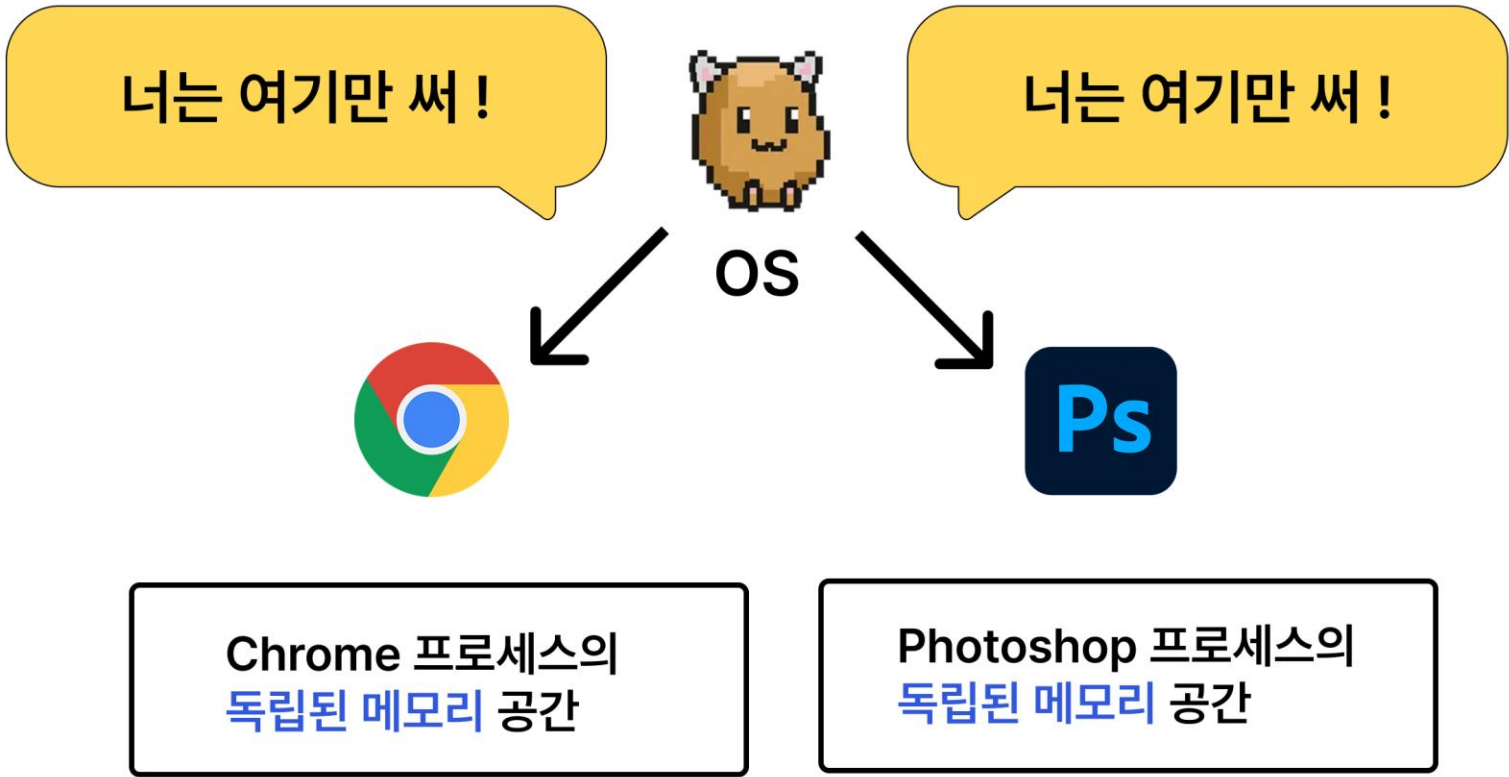
교육 목표

1. 프로세스와 스레드의 정의 파악
2. 스레드 간의 공유, 독립 영역 구분
3. 싱글스레드의 한계, 멀티스레드의 문제점 이해

프로세스 (Process) 란?



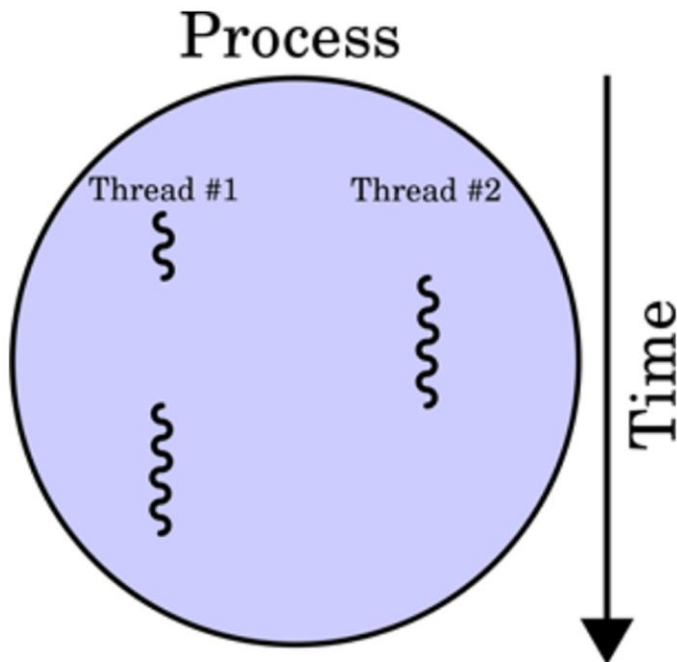
프로세스 = 자원 할당의 단위



프로세스 = 격리 → 안정성 !



스레드 (Thread) 란 ?

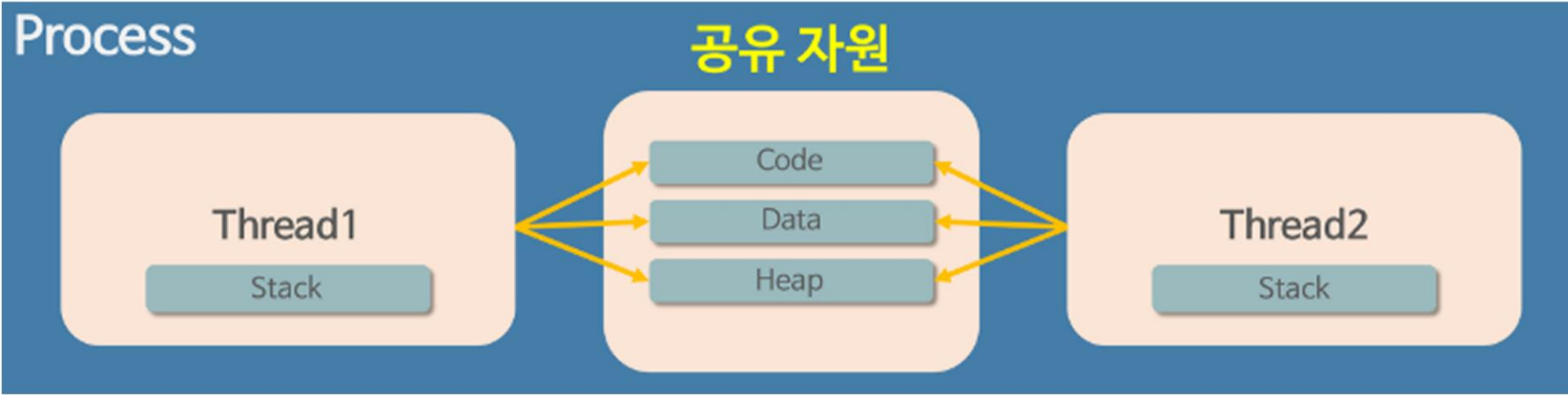


- 프로세스 = 집 (OS가 할당해준 환경)

- 스레드 = 일꾼 (실행의 단위)

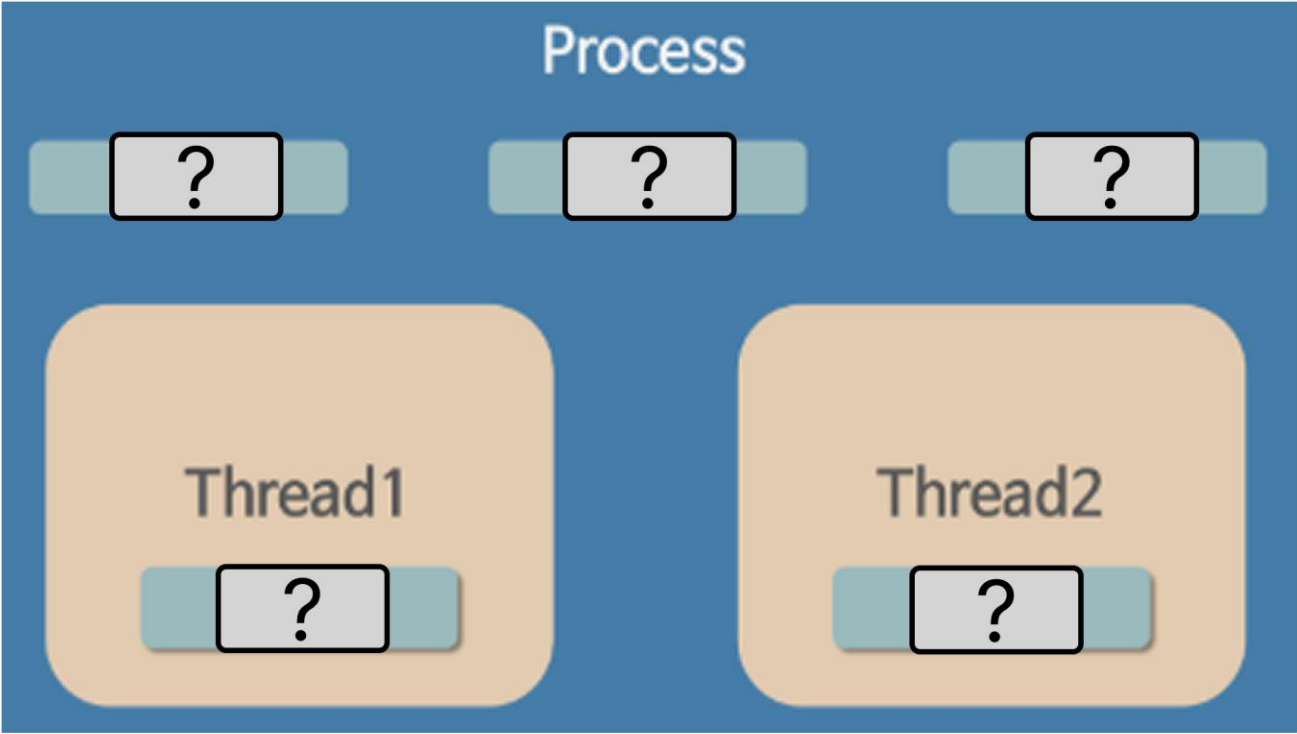
모든 프로세스 : 최소 1개 이상의 스레드를 가짐.

스레드 = 자원 공유

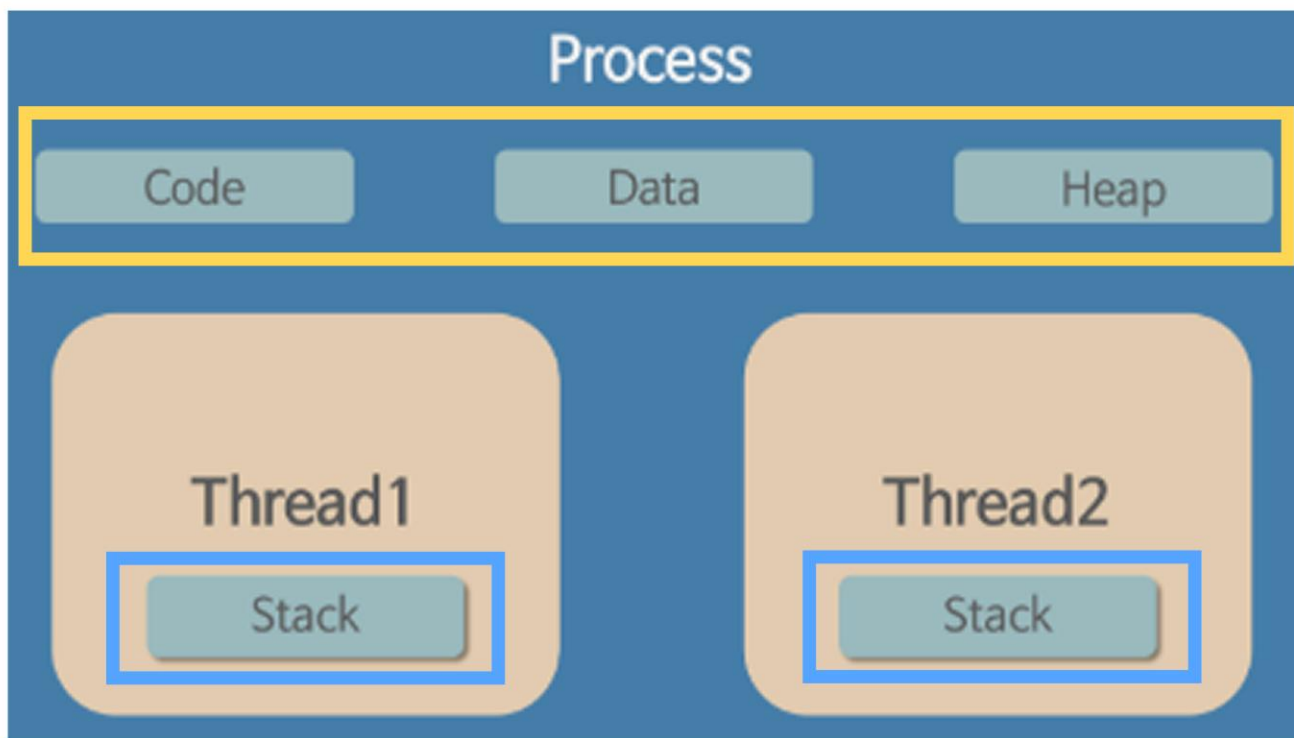


일꾼들(스레드)은 같은 집(프로세스)의 자원을 공유 : '효율성'

메모리 구조 다이어그램



공유 영역 vs 독립 영역



Code: 컴파일된 기계어 코드 저장

Data: 전역, 정적 변수 저장 (시작하면 초기화, 프로세스 종료까지 유지)

Heap: 실행 중 동적할당 변수 저장

Stack: 함수 실행 중 지역변수 저장
임시 공간 (지역, 매개변수, 리턴값 등)

왜 Stack은 독립해야 할까 ?

Stack : 함수의 작업 공간.

```
// T1(일꾼1)이 실행하는 함수
function calculate_A(value) {
  let local_A = value + 10; // T1의 Stack에 저장
}

// T2(일꾼2)가 실행하는 함수
function calculate_B(text) {
  let local_B = text.length; // T2의 Stack에 저장
}
```

일꾼 A(T1): calculate_A() 함수 실행,
일꾼 B(T2): calculate_B() 함수 실행

만약 Stack 공유한다면?
local_A, local_B 변수가 서로 섞임

결론 : 스레드의 '독립적인 실행 흐름'을 위해, Stack은 반드시 각 스레드마다, 독립.

왜 Heap은 공유할까 ?

: '협업'을 위해

```
// Heap에 저장된 '공유 데이터'
shared_news_list = [ "뉴스A", "뉴스B" ]

// T1(일꾼1)이 새 뉴스를 추가
function add_News(news) {
    shared_news_list.append(news); // Heap 데이터 수정
}

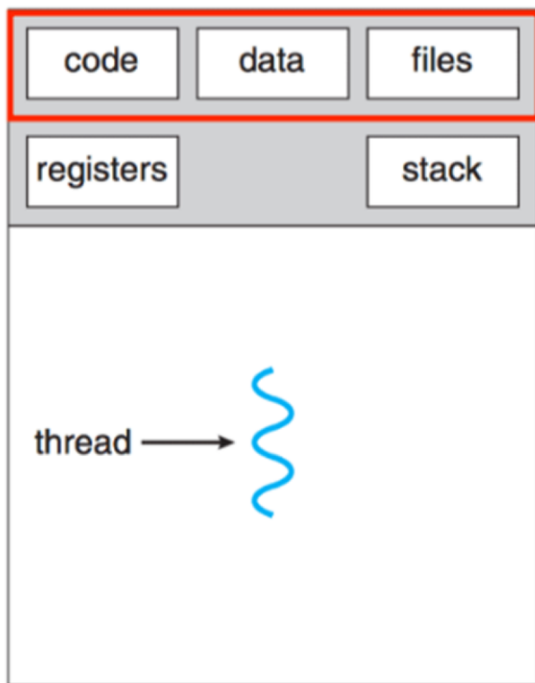
// T2(일꾼2)가 뉴스를 읽음
function get_News_Count() {
    return shared_news_list.length; // Heap 데이터 읽기
}
```

일꾼 A(T1)가 Heap에 '뉴스 데이터'를 가져다 놓으면,
일꾼 B(T2)가 해당 데이터를 꺼내 '화면'에 그려 냄.

이 '공유' 덕분에 스레드들은 효율적으로 협업.

Q. 만약 공유하다 문제가 발생한다면?

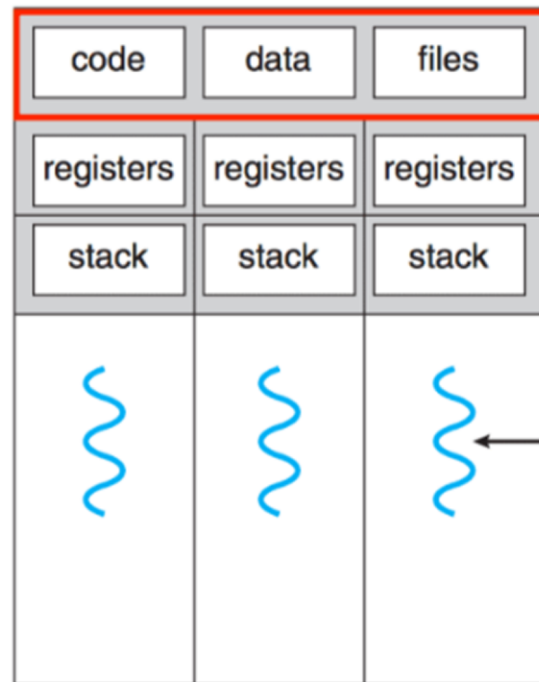
싱글 스레드, 멀티 스레드 란 ?



single-threaded process

프로세스가
스레드 (작업 경로)를
하나만 가지는 방식

한 번에 하나의 작업만
순차적으로 처리



multithreaded process

프로세스가
여러 개의 스레드를
동시에 실행하는 것처럼
보이는 방식

여러 스레드가 프로세스의
자원을 공유하면서
각자 다른 작업을 처리

싱글 스레드의 한계

싱글스레드(작업 수행 경로가 1개)의 문제: 블로킹 I/O

```
import time

def on_download_button_click():
    print("다운로드를 시작합니다...")

    # 이 작업이 10초간 메인 스레드 전체를 멈추게 함
    time.sleep(10) # 10초 걸리는 네트워크 작업이라 가정

    # 10초 뒤에야 이 코드가 실행됨
    print("다운로드 완료!")

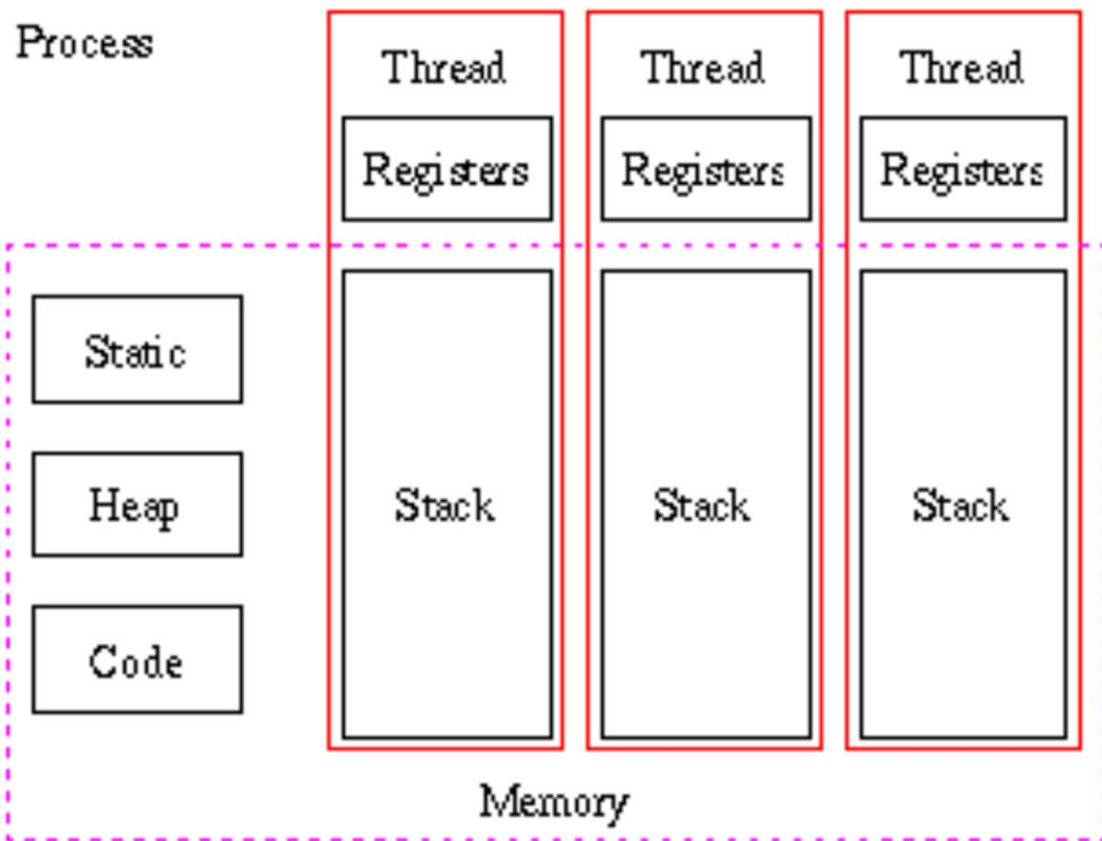
# 메인 스레드가 이 함수를 실행하면,
# 10초 동안 다른 어떤 일(예: 사용자 클릭)도 처리하지 못하고 멈춥니다.
```

블로킹 I/O :
네트워크 요청처럼 **오래 걸리는 작업을 할 때**,
그 작업이 **끝날 때까지 스레드 전체가 멈추는** 현상.

메인 스레드(기본스레드)가 10초간 멈추면,
사용자는 앱이 얼어붙었다고 느낌.

이 문제를 해결하기 위해, **멀티 스레드** 사용

블로킹 I/O 해결, 멀티 스레드의 문제점



멀티 스레드 = 일꾼이 여러 명인 환경

오래 걸리는 I/O 작업 - '워커 스레드'가 처리.

'메인 스레드'는 멈추지 않고 계속해서 사용자의 입력을 처리

여러 일꾼(스레드)이 하나의 공유 자원을 동시에 건드리면?

Heap 공유 등으로 인한 **동시성 문제** 발생.

경쟁 상태 (Race Condition)

스레드 A, B가 동시에 좋아요를 처리한 경우 = (결과는 2가 되어야 함)

```
import threading
import time

total_likes = 0 # 공유 자원 (Heap)

def increment_likes():
    global total_likes

    current_likes = total_likes # 1. 읽기 (A: 0 읽음)
    # (B가 여기서 끼어들어 0을 읽을 수 있음)

    time.sleep(0.001)

    total_likes = current_likes + 1 # 2. 쓰기 (A: 1 씌, B도 1 씌)

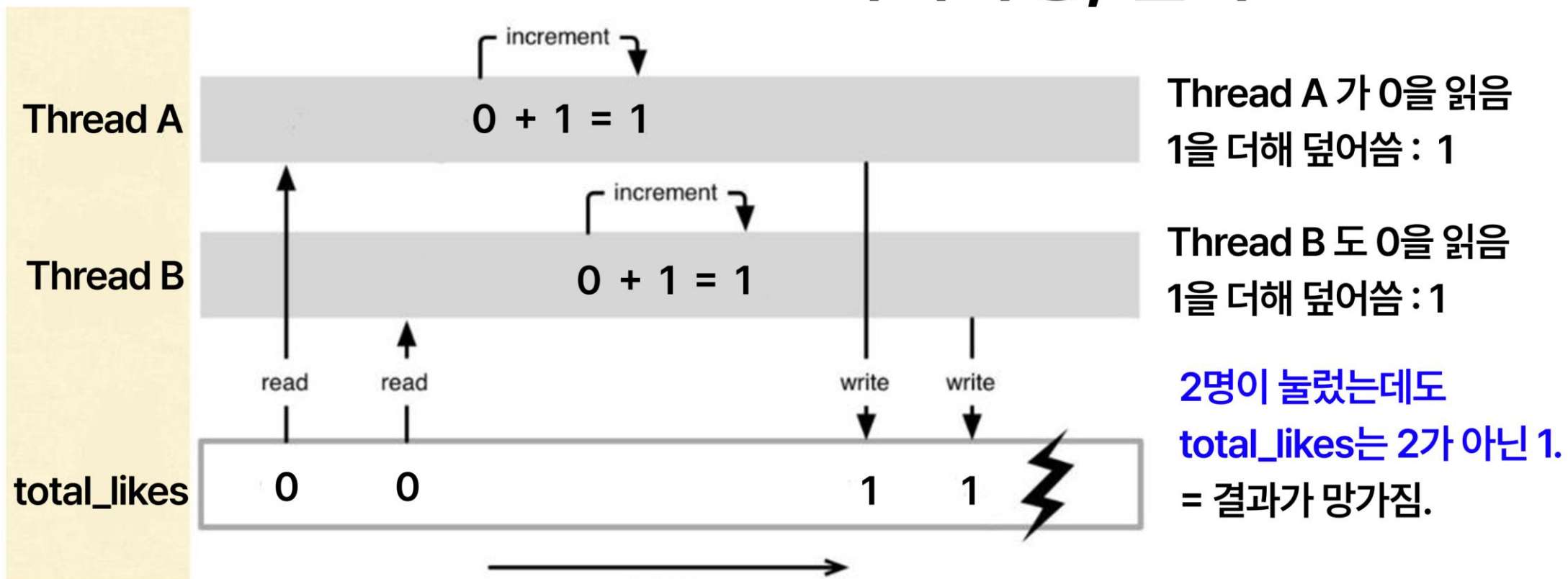
# ... (스레드 2개 생성 및 실행) ...
print(f"최종 좋아요 개수: {total_likes}")
```

공유 자원: total_likes = 0 (Heap에 있는 상황)

increment_likes 함수 안에서,
total_likes를 읽고(read),
1을 더한 뒤(update),
다시 쓰는(write) 작업 수행

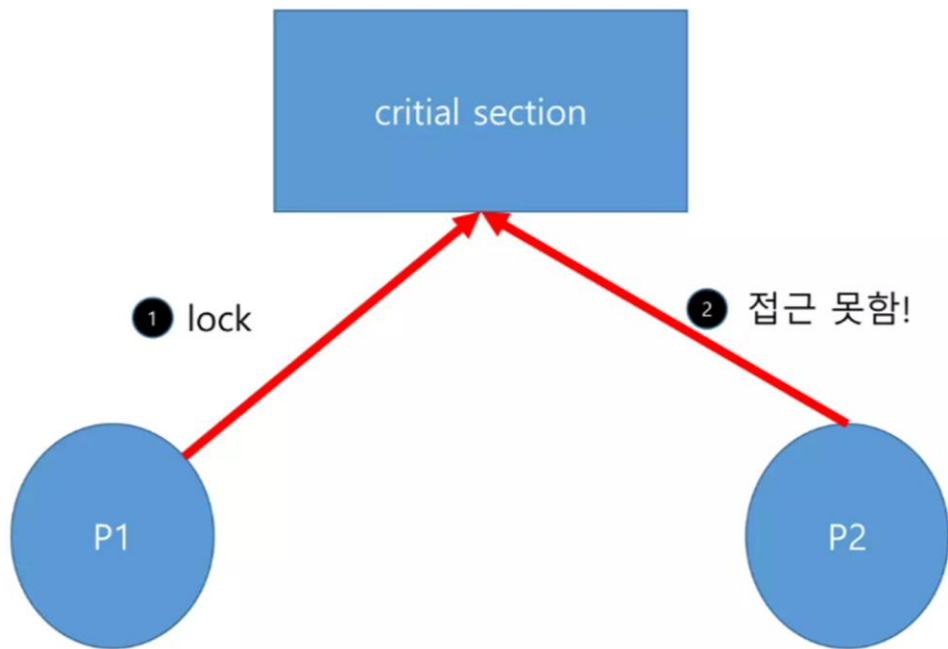
경쟁상태 발생가능한 공유자원 접근 코드 영역
: 임계 구역 (Critical Section)

Race Condition 처리과정, 결과



여러 스레드가 하나의 공용 자원을 동시에 건드려, 결과가 망가지는 경쟁 상태

해결책 : 동기화 (Synchronization)



Mutex (Mutual Exclusion, 상호 배제):

한 스레드가 임계구역(critical section)에 읽고 쓸 동안,
다른 스레드가 접근하지 못하도록 막음.

스레드 A: 1개 공유자원에 접근하기 전, **Lock(잠금)**

다른 스레드들: **대기 (A가 Unlock 할 때까지)**

결과 : 한 번에 하나의 스레드만 데이터를 수정 가능 = 경쟁 상태를 막을 수 있다!

←

→

↺

🏠

🔒

🔗

https://www.cotato.com

🖨

☆

📄

↓

⛶

☰

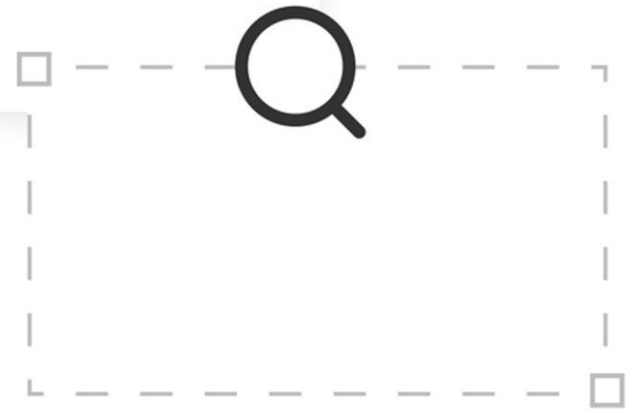
요약

구분	프로세스(Process)	스레드(Thread)
정의	실행 중인 프로그램	프로세스 내 일꾼
핵심	격리(Isolation)	공유(Sharing)
메모리	독립된 메모리 공간	Heap/Data/Code 공유
개인 공간	없음 (프로세스 자체가 단위)	Stack (각자 독립적으로 가짐)

구분	싱글 스레드 (일꾼 1명)	멀티 스레드(일꾼 N명)
작업 방식	순차적 (하나 끝나야 다음 시작)	동시성 (동시에 되는 것처럼 보임)
장점	경쟁 상태 없음	응답성 높음
문제/해결	블로킹 I/O -> 멀티 스레드	경쟁 상태 -> Mutex



◆ CS QUIZ ◆



Q.1

다음 중 프로세스의 정의로 알맞은 것은?

- ① 실행 중인 프로그램의 인스턴스
- ② 프로그램의 실행 흐름 단위
- ③ 현대 CPU 스케줄링 단위
- ④ 프로그램의 전역 변수를 저장하는 영역

A.1

다음 중 프로세스의 정의로 알맞은 것은?

- ① 실행 중인 프로그램의 인스턴스
- ② 프로그램의 실행 흐름 단위
- ③ 현대 CPU 스케줄링 단위
- ④ 프로그램의 전역 변수를 저장하는 영역

정답: 1번

Q.2

하나의 프로세스 내에서 실제로 일을 수행하는 실행의 단위로,
공유된 힙(Heap) 영역을 사용하지만 각자 독립된 스택(Stack)을
가지는 것은 무엇인가요?

- ① 프로세스
- ② 코드
- ③ 스레드
- ④ 데이터

A.2

하나의 프로세스 내에서 실제로 일을 수행하는 실행의 단위로,
공유된 힙(Heap) 영역을 사용하지만 각자 독립된 스택(Stack)을
가지는 것은 무엇인가요?

- ① 프로세스
- ② 코드
- ③ 스레드
- ④ 데이터

정답: 3번

Q.3

운영체제에서 프로세스를 '자원 할당 단위'라고 부르는 이유로
알맞은 것은?

- ① 프로세스마다 CPU와 메모리를 공유하기 때문이다.
- ② 프로세스는 서로의 코드 영역을 자유롭게 접근할 수 있다
- ③ 운영체제가 각 프로세스에 독립된 메모리 공간을 분배하기 때문이다
- ④ 프로세스 내부에 여러 스레드가 포함되기 때문이다

A.3

운영체제에서 프로세스를 '자원 할당 단위'라고 부르는 이유로
알맞은 것은?

정답: 3번

- ① 프로세스마다 CPU와 메모리를 공유하기 때문이다.
- ② 프로세스는 서로의 코드 영역을 자유롭게 접근할 수 있다
- ③ 운영체제가 각 프로세스에 독립된 메모리 공간을 분배하기 때문이다
- ④ 프로세스 내부에 여러 스레드가 포함되기 때문이다

Q.4

[O/X 퀴즈]

프로세스는 자원을 공유하므로,
하나가 오류 나면 다른 프로세스에도 영향을 준다.



A.4

[O/X 퀴즈]

프로세스는 자원을 공유하므로,
하나가 오류 나면 다른 프로세스에도 영향을 준다.

정답: X

Q.5

다음 중, 스레드들이 Heap 영역을 공유하는 가장 핵심적인 이유를 설명한 것은 무엇인가?

- ① 각 스레드의 실행 속도를 높이기 위해
- ② 데이터를 격리하여 안정성을 높이기 위해
- ③ 스레드 간의 효율적인 협업(데이터 공유)을 위해
- ④ 블로킹 I/O 발생 시 프로그램을 멈추기 위해



A.5

다음 중, 스레드들이 Heap 영역을 공유하는 가장 핵심적인 이유를 설명한 것은 무엇인가?

정답: 3번

- ① 각 스레드의 실행 속도를 높이기 위해
- ② 데이터를 격리하여 안정성을 높이기 위해
- ③ 스레드 간의 효율적인 협업(데이터 공유)을 위해
- ④ 블로킹 I/O 발생 시 프로그램을 멈추기 위해

Q.6

[주관식] calculate_A 함수 코드 예시에서, local_A 변수는 프로세스 메모리의 4가지 영역(Code, Data, Heap, Stack) 중 어디에 저장되는가?

```
// T1(일꾼1)이 실행하는 함수
function calculate_A(value) {
  let local_A = value + 10; // T1의 Stack에 저장
}

// T2(일꾼2)가 실행하는 함수
function calculate_B(text) {
  let local_B = text.length; // T2의 Stack에 저장
}
```

A.6

[주관식] calculate_A 함수 코드 예시에서, local_A 변수는 프로세스 메모리의 4가지 영역(Code, Data, Heap, Stack) 중 어디에 저장되는가?

```
// T1(일꾼1)이 실행하는 함수
function calculate_A(value) {
  let local_A = value + 10; // T1의 Stack에 저장
}

// T2(일꾼2)가 실행하는 함수
function calculate_B(text) {
  let local_B = text.length; // T2의 Stack에 저장
}
```

정답: Stack
(또는 스택)

$$\textcircled{4} \quad (1) - / (2) \quad \neg \neg$$

A.7

다음 중 '(1) 프로세스 격리' 와 '(2) 스레드 자원 공유' 중
같은 성격으로 묶인 것을 고르시오.

ㄱ. 카카오톡이 강제로 종료되어도 (크롬)코테이토 사이트의 퀴즈 진행에는
영향이 없다.

ㄴ. 코테이토 사이트 퀴즈에서 문제 로드 스레드가 받아온 데이터를, 동기화
스레드가 같은 메모리 공간에서 참조해 실시간 정답 전송을 수행한다.

① (1) ㄱ / (2) ㄴ

② (1) ㄴ / (2) ㄱ

③ (1) ㄱㄴ / (2) -

④ (1) - / (2) ㄱㄴ

정답: 1번

Q.8

다음은 희정과 자원이의 대화이다. 여기서 자원이가 말한 문제 현상은?

희정: "앱이 갑자기 멈췄어."

자원: "그건 네 앱이 싱글 스레드라서, 네트워크 요청이 끝날 때까지 기다리고 있는 거야."

① 경쟁 상태 (Race Condition)

② 블로킹 I/O

③ Deadlock

④ Context Switching



A.8

다음은 희정과 자원이의 대화이다. 여기서 자원이가 말한 ...
문제 현상은?

정답: 2번

희정: "앱이 갑자기 멈췄어."

자원: "그건 네 앱이 싱글 스레드라서, 네트워크 요청이 끝날 때까지
기다리고 있는 거야."

① 경쟁 상태 (Race Condition)

② 블로킹 I/O

③ Deadlock

④ Context Switching

Q.9

다음 중 멀티스레드를 사용할 때 장점 / 단점으로 알맞은 것은?

- ① 자원 절약 / 메모리 낭비 증가
- ② 독립성 강화 / 스레드 간 통신 어려움
- ③ 응답성 향상 / 동시성(Concurrency) 문제 발생 가능
- ④ cpu 점유율 감소 / 실행 속도 저하



A.9

다음 중 멀티스레드를 사용할 때 장점 / 단점으로 알맞은 것은?

- ① 자원 절약 / 메모리 낭비 증가
- ② 독립성 강화 / 스레드 간 통신 어려움
- ③ 응답성 향상 / 동시성(Concurrency) 문제 발생 가능
- ④ cpu 점유율 감소 / 실행 속도 저하

정답: 3번

Q.10



Race Condition에 관한 개념으로 올바르지 않은 것을 고르시오.

↳ (경쟁 상태)

- ① 임계 구역이란 Race Condition(경쟁 상태)이 발생할 수 있는 코드 영역이다.
- ② Mutex는 한 스레드가 공유자원에 접근중일때 다른 스레드의 접근을 막는 역할을 한다.
- ③ 경쟁 상태(Race Condition)가 발생하면, 그 즉시 프로그램이 오류(Error)를 내며 멈춘다.
- ④ 여러 스레드가 하나의 공용 자원을 동시에 건드려서, 결과가 망가지는 현상이다.



A.10 Race Condition에 관한 개념으로 올바르지 않은 것을 고르시오..
↳ (경쟁 상태)

- ① 임계 구역이란 Race Condition(경쟁 상태)이 발생할 수 있는 코드 영역이다.
- ② Mutex는 한 스레드가 공유자원에 접근중일때 다른 스레드의 접근을 막는 역할을 한다.
- ③ 경쟁 상태(Race Condition)가 발생하면, 그 즉시 프로그램이 오류(Error)를 내며 멈춘다.
- ④ 여러 스레드가 하나의 공용 자원을 동시에 건드려서, 결과가 망가지는 현상이다.

정답: 3번